我用腾讯 Kuikly，把 DeepSeek Harness装进了口袋

原创 腾讯程序员 腾讯技术工程 Sep 8, 2026, 2:57 AM 广东 1231 listened



作者：腾讯程序员yuki

> DSH 这么好玩，但是只能在电脑上玩太可惜了，于是我用 Kuikly 框架做了一个 APP，一码三端的跨端原生 app，按官方 Host 协议跟电脑上的 DeepSeek Harness 对话。

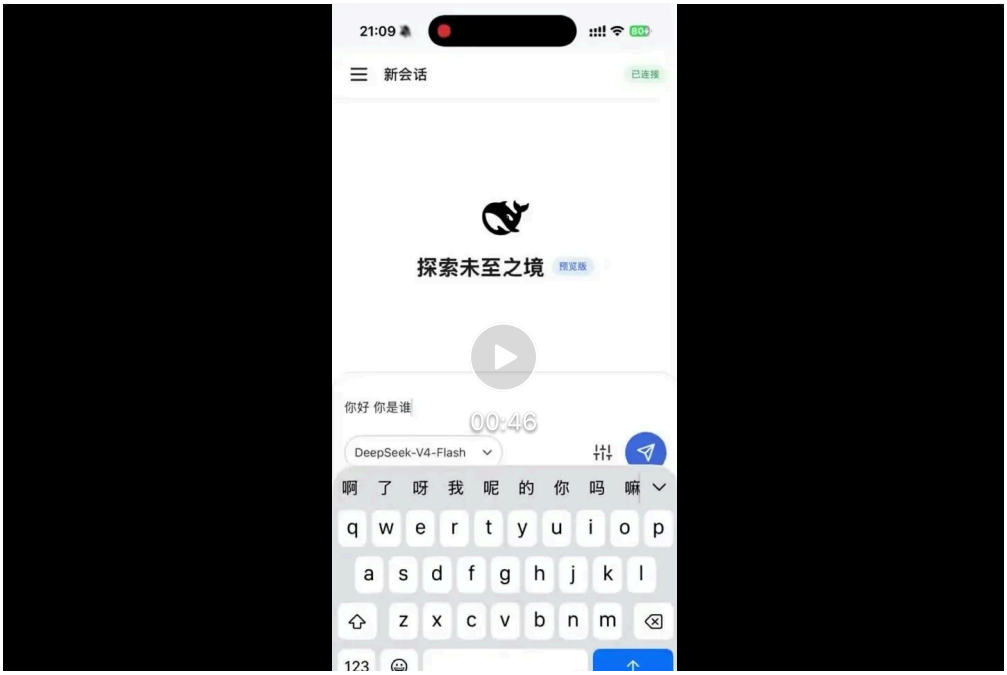
DeepSeek Harness 跑一个任务，经常要几分钟甚至更久。中间它还会停下来等人处理审批、补充信息或确认方向。只要人离开电脑，这一轮任务就可能卡在原地。

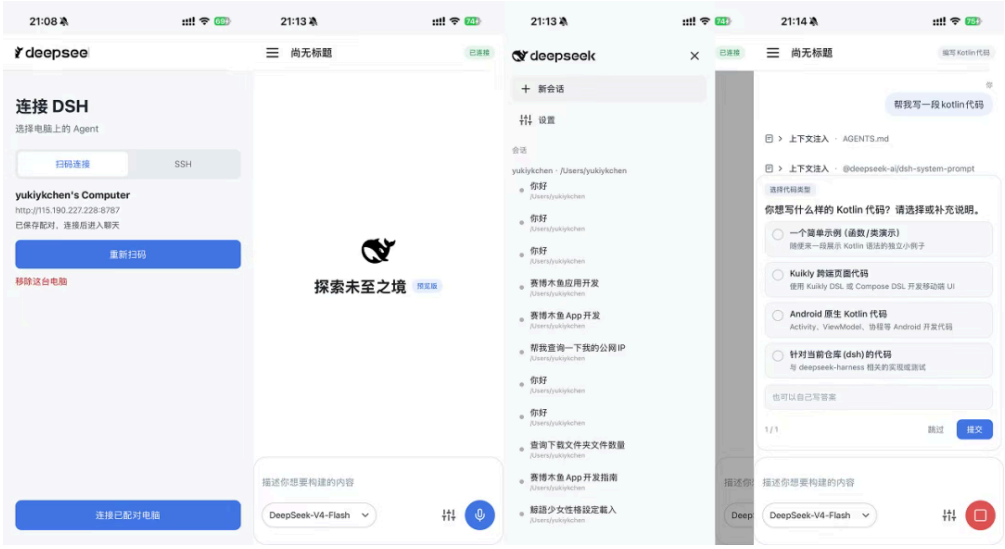
我想解决的不是「在手机上写代码」，而是让手机接住这些短而频繁的交互。通勤、开会或排队时，可以看一眼任务进度，处理一次审批，回答 Agent 的追问，再让电脑继续跑。

于是我用 Kuikly 做了 DSH Mobile。一套 Kotlin 代码覆盖 Android、iOS 和鸿蒙，按照 DSH 官方 Host 协议连接电脑上的 Harness。Agent 循环、工具执行和插件仍在电脑上运行，手机负责连接、交互和渲染。

最省事的方案当然是套一层 WebView，但这个 App 不只是把网页缩进手机。它要维持 WebSocket 长连接，处理锁屏、切后台和网络切换，还要在重连后补回遗漏事件。扫码、SSH 隧道和本地缓存也都需要平台能力。做到这里，原生客户端反而更合适。

DSH App 页面功能演示：





为什么使用 Kuikly

这个项目对跨端框架的要求并不抽象。

聊天正文是持续变化的 Markdown，模型每吐出一个 chunk，页面都要跟着刷新。工具调用、审批卡、提问卡和后台任务会同时更新。底层还有 HTTP RPC、两条 WebSocket 事件流、扫码配对、SSH 隧道和本地存储。三端不仅要长得接近，连接和恢复行为也要一致。

我选择 Kuikly，最直接的原因是开发快。组件市场里已经有不少常用能力，找到合适的组件，加上依赖就能在共享代码里使用，不需要先给 Android、iOS、鸿蒙分别写一套。

Kuikly 的 AI 开发配套也比较实用。我在这个项目里用了 KuiklyUI-AI，它提供 Kuikly DSL 和 Kuikly Compose DSL 的开发规则，也提供组件使用、Module 扩展、网络请求、响应式状态、协程和多端资源等 Skills。这些规则可以交给 CodeBuddy、Cursor、Claude Code 等 AI 编程工具使用。Agent 写页面、接组件或补原生桥接时，不用临时猜 Kuikly 的 API 和工程约定，生成的代码更容易直接落进项目。协议模型、状态机和重复页面可以先交给 Agent 完成，我主要检查交互、平台差异和真机表现。

组件市场：Markdown 和 WebView 快速接入

DSH Mobile 里直接用了两个现成组件 `KuiklyMarkdown` 和 `KuiklyWebview`。

```
implementation("com.tencent.kuiklybase:KuiklyMarkdown:1.0.6-2.1.21")
implementation("com.tencent.kuiklybase:KuiklyWebview:1.0.1-2.0.21")
```

`KuiklyMarkdown` 省掉的工作最多。AI 对话里的 Markdown 不是解析一次就结束，模型输出一直在变，代码块、列表、引用和链接都要跟着刷新。如果从零开始做，需要先找一套三端可用的解析器，再写 AST 到 UI 的映射，处理代码高亮、主题、列表缩进和流式更新。

`KuiklyMarkdown` 已经把这套骨架做好了。它基于 intellij-markdown 解析文本，输出 Block 列表，并提供流式渲染状态。接入以后，我主要处理 DSH 自己的产品逻辑。已经结

束的 Block 保持不动，尾部内容继续更新；代码块暂时没有闭合时，在解析副本里补上闭合标记；页面更新按 16 ms 合帧，避免每来一个小 chunk 就刷新一次。

这样一来，项目不需要自己维护 Markdown 语法解析、代码块渲染和三端基础样式，只需要把精力放在流式场景上。长回答前半部分稳定后不会反复重建，尾部还能继续跟着模型输出刷新。

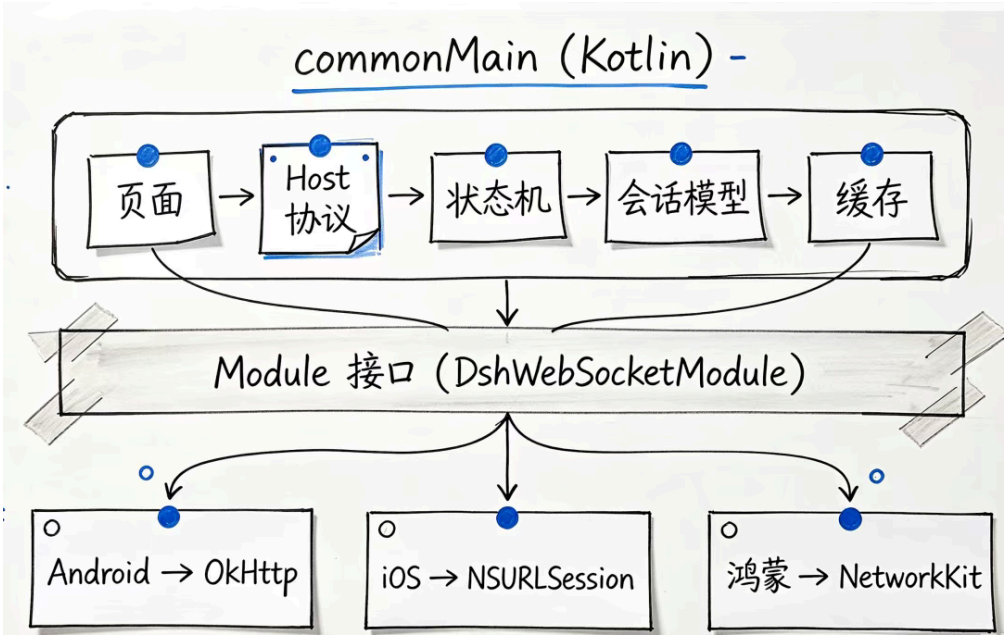
`KuiklyWebview` 用在 Markdown 链接和外部页面的 App 内打开。共享页面里直接放一个 WebView，设置 URL，再监听开始加载、进度、完成和失败事件就够了。刷新和返回逻辑也写在同一份 Kotlin 中。没有这个组件，就要分别封装 Android WebView、iOS WKWebView 和鸿蒙 Web 组件，再把事件桥接回共享层。

组件市场里还有 SQLite、相机、图片选择、录音、定位、蓝牙和 MMKV 等常用能力。这个项目没有把它们全部接进来，但开发新功能时可以先找现成组件。对跨端项目来说，少写的不是一个控件，而是同一项基础能力在三个平台上的接入、对齐和后续维护。

原生桥接能力：差异下沉，共享层保持业务统一

跨端项目真正费时间的地方，通常不是页面，而是 WebSocket、扫码、SSH 和数据库这类系统能力。

DSH Mobile 在 `commonMain` 定义统一的 Module 接口，把连接、发消息、收消息和断开等业务语义固定下来。Android、iOS、鸿蒙分别接自己的系统实现。



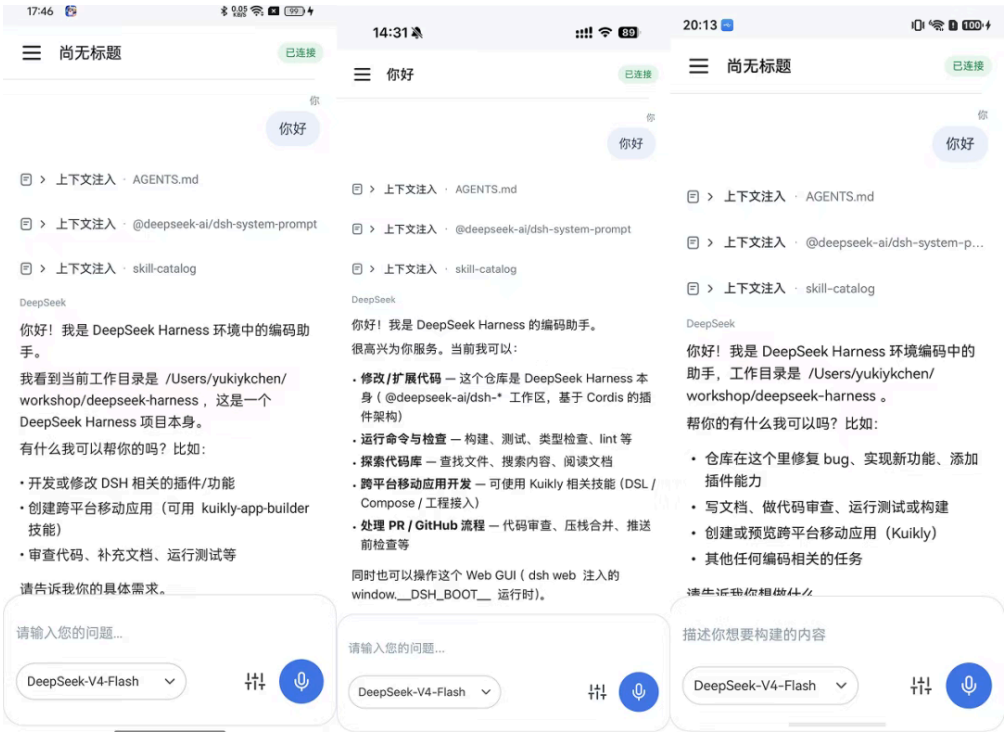
拿 WebSocket 举例，三端底层分别是 `OkHttp`、`NSURLSession`、`NetworkKit`，但上层页面只面对同一个 `DshWebSocketModule`。平台差异被收敛在最底层，聊天页、事件处理、业务状态都不需要知道自己在哪个系统上。SSH 隧道、扫码配对是同一套拆法。

这层下沉的收益会随协议变化放大，DSH 还没有完全稳定下来，往后每多一个 RPC、改一个事件，只需改共享层，三端一起生效，这比到时候再维护三个壳便宜得多。

目前项目约有 1.3 万行 `commonMain` Kotlin 代码由三端共享。Android、iOS、鸿蒙宿主各有三四千行，主要用于接入系统能力和完成平台工程配置。组件能直接复用，协议和状态又

留在共享层，不用把相同功能在三个工程里各做一遍。

Kuikly 在三端同一页面的表现：



安卓、iOS、鸿蒙三端UI对比页面

不加中间层，直接接 Host 协议

DSH Mobile 没有在手机和电脑之间再放一套业务服务。它直接使用官方 Web 前端背后的 Host 协议。浏览器如何调用 DSH，App 就沿用同一套方法和事件。

DSH 本身是基于 Cordis 的插件化 Agent 运行时。与移动端连接直接相关的部分可以简化成三层。`core/session`、`agent-loop` 和 `tools` 负责会话、Agent 循环与工具执行，并产生相应事件。`host/apiproxy` 把内部能力整理成 RPC 方法表和两条下行事件流。`client/connection` 再把这些接口接到本机的 3080 端口，对外提供 HTTP `/api/...`、WebSocket `/api/events.mux` 和 `/api/events.host`。

电脑上的 Agent 继续读写工作区、调用工具和运行插件。App 只连接最外层的 `client/connection`，不需要理解或重新翻译 DSH 内部的插件结构。

发消息就是一次 HTTP RPC

DSH 把可调用能力登记在一张 RPC 方法表里。以 `dsh-v0.1.1-rc.2` 为例，表中有 52 个方法。发送消息对应 `session.prompt`，请求路径是 `POST /api/session.prompt`。会话列表、历史记录、工作区、模型、Goal 和设置也都走同一条 RPC 通道。

方法的入参和返回值来自 TypeScript 函数签名，注册表负责把方法名映射到实际实现。DSH Mobile 只实现当前需要的部分，没有为移动端另造一套接口。

完整方法表可以直接看 rpc-map.ts。52 这个数字只对应文中使用的版本。DSH 仍在快速迭代，接入时最好锁定已经验证过的 tag，不要默认 master 始终兼容。

两条 WebSocket，各管一类状态

远程连接建立后，Host 会通过两条只下行的 WebSocket 向 App 推送事件。

`/api/events.mux` 负责当前会话中正在发生的事，包括模型输出、工具调用、审批、提问、消息队列和后台任务。

`/api/events.host` 负责电脑这一侧的全局变化，包括会话增删、运行状态、工作区变更和 Host 级错误。

两条流分开以后，会话正文和全局状态不会挤在同一个通道。App 也可以按不同的生命周期处理它们。聊天页面主要消费 mux，工作区和会话列表更多依赖 host。

有两个细节需要单独处理。

`session/queue` 和 `session/jobs` 推送的是完整快照，不是「又增加了一条」的增量事件。它们不进入会话日志，重连时也不能只靠事件回放恢复，所以 App 收到新快照后会直接覆盖本地状态。

`session/projection.value` 和 `host/remote-event.args` 在 carrier 层是宽类型。它们的结构由各自业务包负责，App 不能假设所有帧都严格符合固定 data class。这里需要保留未知字段，并在解析失败时降级，而不是让整条事件流一起中断。

断线以后，先补事件，再对历史

移动网络一定会断。锁屏、切后台、进隧道、Wi-Fi 与蜂窝网络切换都可能让连接失效。

DSH Mobile 重连后按下面的顺序恢复。

1. 重新建立 SSH 或 Relay 隧道。
2. 使用上次收到的序号补回遗漏的 `session/event`。
3. 请求一次 `session.history`，重新对齐聊天记录。
4. 使用最新快照覆盖 queue 和 jobs。

每次连接还带一个世代号。连接断开后，旧连接中迟到的 RPC 响应会被丢弃，不能再写进新的会话状态。

如果 Agent 在断线期间仍在电脑上运行，App 重连后会重新订阅这一轮任务，而不是把用户的 Prompt 再发一次。这个区别很重要。重连是恢复观察和控制，不是重新执行任务。

状态机位于共享层，因此三端采用同样的补齐顺序和失效规则。平台代码只负责把底层连接事件交给共享层。

DSH Mobile 的两种远程连接方式

DSH 默认监听 `127.0.0.1:3080`。这是应该保留的默认值，因为能访问这个端口的客户端，可能通过 Agent 和工具获得很高的本机操作权限。DSH Mobile 没有直接把 3080 开到公网，而是提供 SSH 和扫码 Relay 两种连接方式。

有 SSH 就直接打隧道

SSH 模式在手机上建立本地端口转发，把 App 侧的一个 loopback 端口映射到电脑的 `127.0.0.1:3080`。HTTP RPC 和两条 WebSocket 都通过这条隧道传输。

这种方式没有改变 DSH 的认证逻辑。认证发生在 SSH 层，DSH 仍然只看到来自本机回环的请求。已经有 SSH 主机和密钥配置时，这条路径最直接。

不想配 SSH，扫个码也能连

手动填写地址、端口和密钥不适合普通移动端操作，所以我另外做了 dsh-scan-remote 插件。

插件安装后，DSH Settings 中会出现 Remote Access 页面。电脑生成二维码，手机扫描后完成配对。电脑端插件和 App 都主动连接 Relay，再通过 `sealed-tunnel-v1` 转发 DSH 流量。

连接时，DSH 仍然只监听电脑上的 `127.0.0.1:3080`。Host 插件从本机访问 DSH，并主动连接 Relay；手机 App 也主动连接同一个 Relay。两端完成配对后，App 发出的 HTTP 和 WebSocket 流量会经过密封隧道转发到 Host 插件，再由插件送到本机的 DSH。整个过程中，Relay 不需要直接访问 3080 端口，电脑也不需要把 DSH 服务开放到局域网或公网。

二维码中的主密钥位于 URL fragment 中，正常 HTTP 请求不会把 fragment 发给 Relay。隧道数据经过密封后再转发，Relay 只负责连接双方和传递数据。

把手机作为决策节点入口

DSH Mobile 目前更适合短、轻、高频的交互。

Agent 运行时，可以在手机上查看流式回复和工具状态。遇到命令审批时，直接允许或拒绝。Agent 提问时，补一句信息让任务继续。后台 jobs、Goal 进度和会话状态也适合在碎片时间里查看。

长 Prompt、大段代码 Diff 和需要持续思考的修改仍然更适合桌面。手机屏幕和输入方式没有因为接入 Agent 就突然变大。这个 App 的定位是远程控制面板，不是把完整 IDE 搬到手机上。

这类产品现在已经越来越常见。Cursor 的 iOS 应用 可以启动云端 Agent，也能通过 Remote Control 操作电脑上的 Agent。ChatGPT 的桌面版说明 是 Codex 已经进入 ChatGPT 移动应用可以继续 Mac 主机上正在运行的任务。Claude Code Remote Control 也支持从手机接续本机会话。

几家的执行位置和数据路径并不相同，但它们都在处理同一个变化。具体编码工作交给 Agent 后，人更多是在任务节点上给意图、做判断和看结果。这些动作本来就不要求人一直坐在电脑前。

DSH Mobile 下一步会先补输入能力。`session.prompt` 已经支持图片内容数组，可以在此基础上增加图片上传和语音输入。通知能力也需要电脑端配合，单纯给 App 申请通知权限并不能知道任务何时需要人处理。现阶段不会为了推送而私自增加官方协议之外的 RPC，先把连接和重连稳定性处理好。

这套客户端也不必永久绑定 DSH。对多数 Agent Host 来说，移动端需要的基础能力很接近，包括列出会话、下发消息、接收事件和处理审批。流式 Markdown、工具卡片、断线状态机与会话模型已经放在共享层。以后接其他 Agent 服务时，主要新增协议适配器，不需要重新写三端界面。

快速跑起来

纸上得来终觉浅，绝知此事要躬行。说了这么多，还得是自己上手体验一下效果。

启动 DSH 和 Relay

本地扫码连接可以按四步完成。

1. 启动 Relay。
2. 安装扫码插件并启动 DSH。
3. 在手机上安装 DSH Mobile。
4. 扫描 Settings 中的 Remote Access 二维码。

先让手机和电脑连接同一个可信 Wi-Fi 或热点。

```
# 终端 1，启动本地 Relay
git clone https://github.com/yukiykchen/dsh-scan-remote.git
cd dsh-scan-remote/relay
cp .env.example .env
npm ci
npm run build
HOST=0.0.0.0 PORT=8787 npm start
```

局域网试用时，Relay 需要监听手机能够访问的网络接口。`HOST=0.0.0.0` 会开放 8787 端口到本机所有网络接口，请只在可信网络中使用，并检查系统防火墙设置。

然后安装插件并启动 DSH。

```
# 终端 2，安装插件并启动 DSH
npx @deepseek-ai/dsh plugin --profile web add \
  "github:yukiykchen/dsh-scan-remote#v0.0.1"

export PUBLIC_RELAY_URL=http://192.168.1.10:8787
npx @deepseek-ai/dsh web
```

`PUBLIC_RELAY_URL` 要换成电脑当前的局域网地址。它会写进二维码，必须能从手机访问。DSH 启动后，打开 Settings，再进入 Remote Access 页面。



下载和安装 App

Android 可以从 releases 页面 下载 APK 后安装。

iOS 需要使用 Xcode 打开 `iosApp/iosApp.xcworkspace`，配置自己的开发者签名后安装到真机。鸿蒙端可用 DevEco Studio 打开 `ohosApp` 自行构建。代码仓库链接：deepseek-harness-mobile。

如果扫码后一直超时，先检查三件事。

- 手机能否访问 `http://电脑局域网地址:8787/health`。
- `PUBLIC_RELAY_URL`
  是否仍是电脑当前地址。
- Relay 是否真的监听了手机可达的接口，防火墙是否允许 8787 端口。

电脑更换 Wi-Fi 或热点后，局域网地址通常会变化。此时需要更新 `PUBLIC_RELAY_URL`，重启 DSH，再扫描新二维码。

腾讯云上运行：让 Agent 不必依赖电脑在线

除在个人电脑本地运行外，DSH Mobile 也可以配合腾讯云轻量应用服务器使用。将 DeepSeek Harness 部署在轻量应用服务器上后，Agent 循环、工具执行、插件和工作区都持续运行在云端；手机 App 仍然只负责连接、交互和渲染。

这样，长任务不再依赖个人电脑一直开机。即使人离开工位或合上电脑，Harness 也可以继续执行；用户则可以在手机上查看流式输出和工具状态，在需要时处理审批、补充上下文或确认下一步方向。

想继续开发，从这几个入口动手

如果只想扩展 App，入口比较集中。

- 新增 Host 方法时，在 `DshHostProtocol.kt` 增加定义，然后复用现有 RPC 通道。只要不涉及新的系统能力，三端不需要分别修改原生代码。
- 新增原生能力时，先在 `commonMain` 定义 Module，再补三端实现。Android 从 `KuiklyRenderActivity` 导出模块，iOS 实现放在 `KuiklyExpand/Modules`，鸿蒙实现放在 `kuikly/modules`。
- 新增页面时，创建带 `@Page` 注解的页面类，KSP 会生成路由。
- 如果要让 DSH 内部的新事件出现在手机上，可以写一个 DSH 插件监听对应事件，并通过 `host/remote-event` 转发。具体事件仍要加入 Host 侧允许转发的范围，App 端也要对宽类型参数做容错。

注册新的官方 RPC 方法或替换整套传输层，就不再是单纯修改 App，而是需要改 DSH 本体。此类改动更适合按上游项目的方式提交和维护。

相关仓库与链接

- Kuikly 框架
- Kuikly 组件仓库
- DSH Mobile
- 扫码插件与 Relay
- KuiklyUI AI 开发规则与 Skills
- 腾讯云轻量应用服务器
- DSH RPC 方法表
- DSH 下行事件定义

DSH 仍处于 developer preview，协议和包结构都可能发生破坏性变化。本文中的方法数量、目录和事件名称对应 dsh-v0.1.1-rc.2。实际开发时应锁定已经验证过的版本，再逐步跟进上游变化。

关于 Kuikly

Kuikly 是腾讯开源的高性能跨端框架，基于 Kotlin Multiplatform 技术，覆盖 Android、iOS、HarmonyOS、H5、微信小程序、Mac 六大平台，支撑业务日活用户超5亿。

当前Kuikly已经开源，有兴趣和有需要的产品，可以通过以下方式访问 Kuikly 仓库和文档，欢迎Star、Watch与体验：

👉 Github 仓库 | 📚 官方文档

Kuikly框架属于腾讯端服务联盟（tds.qq.com）的重要成员，欢迎关注及了解更多信息：

- 腾讯端服务官网： TDS 腾讯端服务
- Kuikly官网：跨平台框架

腾讯端服务服务号	Kuikly 公众号	在线咨询





阅读 3.0万

留言

写留言

还没有留言，来写首评